মেশিন লার্নিং-এ ক্যালকুলাস (Partial Derivatives, Gradient, Gradient Descent)

নিচের অংশটি মেশিন লার্নিং-এ ক্যালকুলাস কেন দরকার — সেটি ব্যাখ্যা করা হলো।

1. মেশিন লার্নিংয়ের মূল ধারণা: Error কমানো

যখন আমরা একটি Machine Learning model train করি, তখন মূল লক্ষ্য হলো:

Prediction error যতটা সম্ভব কম করা।

এই error মাপার জন্য আমরা একটি ফাংশন ব্যবহার করি যাকে বলে:

Loss Function

ধরা যাক:

$L(w)$

এখানে
L = Loss (Error)
w = model এর weights

একটি মডেলে হাজার বা লাখ weights থাকতে পারে।

উদাহরণ:

$w = [w_1, w_2, w_3, \ldots, w_{\square}]^T$

অর্থাৎ
w_1 = প্রথম weight
w_2 = দ্বিতীয় weight
…
$w_{\square}$ = n-তম weight

Loss function নির্ভর করে এই সব weight এর উপর।

2. Linear Algebra কীভাবে ব্যবহার হয়

Machine Learning এ data সাধারণত vector বা matrix আকারে থাকে।

উদাহরণ:

Data vector:

$x = [x_1, x_2, x_3]$

Weights vector:

$w = [w_1, w_2, w_3]$

Prediction সাধারণত হয়:

$y = w^T x$

এটা Linear Algebra দিয়ে করা হয়।

অর্থাৎ:

- data representation
- vectors
- matrices
- matrix multiplication

সব Linear Algebra দ্বারা পরিচালিত।

তাই বলা হয়:

Linear algebra dictates how data is represented.

3. কিন্তু Model কীভাবে শেখে?

ডেটা representation আলাদা বিষয়।

কিন্তু model শেখে calculus ব্যবহার করে।

কারণ training মানে হচ্ছে:

Loss minimize করা

অর্থাৎ error কমানো।

এখানেই আসে:

- derivatives

- gradients
- gradient descent

তাই বলা হয়:

Calculus dictates how models learn.

4. Single Variable Calculus

ধরা যাক একটি function:

$f(x)$

যদি আমরা minimum খুঁজতে চাই তাহলে দেখি slope কোথায় zero।

$f'(x) = 0$

যেখানে slope = 0
সেখানে minimum বা maximum হতে পারে।

এটি একটি variable এর calculus।

5. Machine Learning এ সমস্যা

Machine learning এ function হয়:

$L(w_1, w_2, w_3, \ldots, w_{\square})$

এখানে

- একাধিক variable আছে
- হাজার হাজার weight থাকতে পারে

অর্থাৎ function টি হলো:

multi-variable function

এটি একটি high-dimensional surface তৈরি করে।

6. Landscape ধারণা

Loss function কে একটি landscape হিসেবে ভাবা যায়।

ধরা যাক:

- পাহাড়
- উপত্যকা
- ঢাল

Loss landscape এর মধ্যে:

- উপরে = বেশি error
- নিচে = কম error

আমাদের লক্ষ্য:

সর্বনিম্ন জায়গা (global minimum) খুঁজে পাওয়া।

7. Partial Derivative কী?

যেহেতু অনেক variable আছে, তাই প্রতিটি weight এর জন্য আলাদা derivative নিতে হয়।

উদাহরণ:

$L(w_1, w_2)$

তাহলে:

$\partial L/\partial w_1$

মানে
w_1 একটু পরিবর্তন করলে loss কত পরিবর্তন হয়

আর

$\partial L/\partial w_2$

মানে
w_2 পরিবর্তন করলে loss কত পরিবর্তন হয়

এই derivative গুলোকে বলে:

Partial Derivatives

8. Gradient Vector

সব partial derivative একসাথে লিখলে আমরা পাই:

$$\nabla L = \begin{bmatrix} \partial L/\partial w_1 \\ \partial L/\partial w_2 \\ \vdots \\ \partial L/\partial w_n \end{bmatrix}$$

এটাকে বলে:

Gradient Vector

Gradient vector বলে দেয়:

Loss কোন দিকে দ্রুত বাড়ছে।

9. Gradient Descent

Gradient বলে দেয় কোন দিকে error বাড়ছে।

কিন্তু আমরা চাই error কমাতে।

তাই আমরা gradient এর উল্টো দিকে যাই।

Update rule:

$$w = w - \eta \nabla L$$

এখানে
w = weight
η = learning rate
∇L = gradient

অর্থাৎ:

নতুন weight = পুরোনো weight − learning rate × gradient

10. সহজভাবে পুরো বিষয়টি

Machine Learning training এর ধাপ:

1. model prediction করে
2. loss calculate হয়
3. partial derivatives বের করা হয়
4. gradient vector তৈরি হয়
5. gradient descent দিয়ে weights update হয়

6. error ধীরে ধীরে কমে

11. একটি বাস্তব উদাহরণ (intuition)

ভাবো তুমি একটি পাহাড়ে দাঁড়িয়ে আছো।

তুমি চোখ বন্ধ করে আছো।

তুমি জানতে চাও নিচে কোথায় নামলে সবচেয়ে দ্রুত নামা যাবে।

Gradient ঠিক সেটাই বলে:

কোন দিকে ঢাল সবচেয়ে বেশি।

Gradient descent মানে:

সেই ঢালের উল্টো দিকে হাঁটা যাতে নিচে পৌঁছানো যায়।

2. Partial Derivatives: একটি ভ্যারিয়েবল আলাদা করে দেখা

Partial derivative এমন একটি derivative যা দেখায়—একটি বহু-ভ্যারিয়েবল ফাংশনে একটি ভ্যারিয়েবল সামান্য পরিবর্তন করলে ফলাফল কত পরিবর্তন হয়, যখন অন্য সব ভ্যারিয়েবল স্থির রাখা হয়।

এটি বোঝাতে আমরা ব্যবহার করি **∂ (curly d)** চিহ্ন।

1. Partial Derivative কী মাপে

ধরা যাক একটি ফাংশন:

f(x, y)

এখানে দুটি ভ্যারিয়েবল আছে:

- x
- y

এখন যদি আমরা জানতে চাই:

শুধু x পরিবর্তন করলে f কত পরিবর্তন হবে, তখন আমরা নেই:

∂f/∂x

এখানে:

- x পরিবর্তন হবে
- y স্থির থাকবে

2. Partial Derivative এর গাণিতিক সংজ্ঞা

$$\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x+h, y) - f(x, y)}{h}$$

এর অর্থ:

1. x কে সামান্য h পরিমাণ বাড়াও
2. নতুন ফাংশনের মান বের করো
3. পুরনো মান বাদ দাও
4. পরিবর্তনকে h দিয়ে ভাগ করো
5. তারপর h কে 0 এর দিকে নিয়ে যাও

এটি মূলত slope মাপার পদ্ধতি।

3. Partial Derivative কিভাবে বের করা হয় (Mechanics)

Partial derivative বের করার নিয়ম খুব সহজ।

Rule:

যে ভ্যারিয়েবলের respect-এ derivative নিচ্ছো
অন্য সব ভ্যারিয়েবলকে constant ধরে নাও।

অর্থাৎ তাদেরকে সংখ্যা (2, 5, 10) এর মতো ধরতে হবে।

4. উদাহরণ

ধরা যাক:

$f(x, y) = x^2y + \sin(x) + e^y$

এখানে তিনটি term আছে:

1. x^2y
2. $\sin(x)$
3. e^y

5. $\partial f/\partial x$ বের করা

এখন আমরা বের করব:

$\partial f/\partial x$

এখানে y কে constant ধরা হবে।

Step-by-step

Term 1:

x^2y

y constant, তাই

$d/dx\ (x^2y) = 2xy$

Term 2:

$\sin(x)$

Derivative:

$\cos(x)$

Term 3:

e^y

এখানে x নেই, তাই derivative হবে:

0

Final result:

$\partial f/\partial x = 2xy + \cos(x)$

6. $\partial f/\partial y$ বের করা

এখন বের করব:

$\partial f/\partial y$

এখানে x কে constant ধরা হবে।

Term 1:

x^2y

Derivative:

x^2

Term 2:

$\sin(x)$

এখানে y নেই, তাই:

0

Term 3:

e^y

Derivative:

e^y

Final result:

$\partial f/\partial y = x^2 + e^y$

7. Machine Learning এ এর অর্থ

ধরা যাক:

f = Loss

Loss নির্ভর করছে weights এর উপর।

$L(w_1, w_2, w_3, \ldots)$

এখন

$\partial L/\partial w_1$

মানে:

weight w_1 একটু পরিবর্তন করলে loss কত পরিবর্তন হবে।

উদাহরণ:

ধরা যাক

$\partial L/\partial w_1 = 5$

এর মানে:

w_1 সামান্য বাড়ালে loss দ্রুত বাড়বে।

তাই training এর সময়:

w_1 কমানো হবে।

8. কেন Partial Derivative গুরুত্বপূর্ণ

Machine Learning এ:

- হাজার হাজার weights থাকে
- প্রতিটি weight error কে আলাদা ভাবে প্রভাবিত করে

তাই আমাদের জানতে হয়:

- কোন weight বেশি error বাড়াচ্ছে
- কোন weight কম প্রভাব ফেলছে

এই তথ্য দেয়:

Partial derivatives

9. Gradient এর সাথে সম্পর্ক

সব partial derivative একসাথে করলে পাই:

$$\nabla L = \begin{bmatrix} \partial L/\partial w_1 \\ \partial L/\partial w_2 \\ \vdots \\ \partial L/\partial w_{\square} \end{bmatrix}$$

এটাই Gradient Vector।

Gradient ব্যবহার করেই চলে:

Gradient Descent Algorithm

3. Gradient Vector (∇f)

যখন আমরা একটি বহু-ভ্যারিয়েবল ফাংশনের সব **partial derivatives** একসাথে একটি **vector** এ রাখি, তখন তাকে বলা হয় Gradient Vector।

Gradient বোঝাতে সাধারণত ব্যবহার করা হয় ∇ (nabla) চিহ্ন।

ধরা যাক একটি ফাংশন:

$f(w_1, w_2, \ldots, w_n)$

এখানে:

$w_1, w_2, \ldots, w_n$ = বিভিন্ন variables
(machine learning এ এগুলো সাধারণত weights)

Gradient হবে:

$$\nabla f = \begin{bmatrix} \partial f/\partial w_1 \\ \partial f/\partial w_2 \\ \vdots \\ \partial f/\partial w_n \end{bmatrix}$$

অর্থাৎ:

Gradient vector = সব partial derivative একসাথে

3.1 Gradient এর সবচেয়ে গুরুত্বপূর্ণ geometric property

Gradient vector এর একটি খুব গুরুত্বপূর্ণ বৈশিষ্ট্য আছে।

Gradient সবসময় সেই দিকে নির্দেশ করে যেখানে ফাংশন সবচেয়ে দ্রুত বাড়ে।

অর্থাৎ:

direction of steepest ascent

সহজভাবে:

- কোন দিকে গেলে function সবচেয়ে দ্রুত increase করবে
- Gradient সেই দিক দেখায়

পাহাড়ের উদাহরণ

ভাবো তুমি একটি পাহাড়ের ঢালে দাঁড়িয়ে আছো।

চারদিকে অনেক দিক আছে যেখানে হাঁটা যায়।

কিন্তু একটি দিক আছে যেখানে:

সবচেয়ে দ্রুত উপরে ওঠা যায়।

Gradient সেই দিকটি নির্দেশ করে।

Gradient এর magnitude (length)

Gradient vector এর একটি length বা magnitude থাকে।

এটি লেখা হয়:

$||\nabla f||$

এটি বলে:

ঢাল কতটা খাড়া।

অর্থাৎ:

- gradient ছোট → slope কম
- gradient বড় → slope বেশি

Machine Learning এ এর অর্থ

ধরা যাক:

f = Loss

Gradient বলবে:

কোন দিকে গেলে loss দ্রুত বাড়বে।

কিন্তু আমরা চাই:

loss কমাতে।

তাই আমরা gradient এর উল্টো দিকে যাই।

এটাই:

Gradient Descent

3.2 Directional Derivative

এখন একটি গুরুত্বপূর্ণ প্রশ্ন আসে।

ধরা যাক gradient যে দিক দেখাচ্ছে সেই দিক না গিয়ে আমরা অন্য কোনো নির্দিষ্ট দিকে চলতে চাই।

তাহলে সেই দিকে slope কত?

এটা বের করার জন্য ব্যবহার করা হয়:

Directional Derivative

ধরা যাক একটি direction vector আছে:

u

এটি একটি unit vector (length = 1)

Directional derivative:

$D_u f = \nabla f \cdot u$

এখানে:

∇f = gradient
u = direction
$\cdot$ = dot product

Dot Product Formula

Dot product expand করলে পাই:

$D_u f = ||\nabla f||\ ||u|| \cos(\theta)$

এখানে:

$||\nabla f||$ = gradient এর magnitude
$||u||$ = direction vector এর magnitude
θ = gradient এবং direction vector এর মাঝে angle

কেন **Gradient** সবচেয়ে খাড়া দিক

$\cos(\theta)$ এর মান:

$\theta = 0° \rightarrow \cos(\theta) = 1$
$\theta = 90° \rightarrow \cos(\theta) = 0$
$\theta = 180° \rightarrow \cos(\theta) = -1$

Directional derivative সর্বোচ্চ হবে যখন:

$\cos(\theta) = 1$

অর্থাৎ:

$\theta = 0$

মানে:

direction vector gradient এর সাথে একই দিকে।

তাই প্রমাণ হয়:

Gradient direction = steepest increase

Machine Learning এর জন্য এর গুরুত্ব

এই property এর কারণেই:

Gradient Descent কাজ করে।

কারণ:

1. Gradient বলে দেয় loss কোন দিকে সবচেয়ে দ্রুত বাড়ছে
2. আমরা gradient এর উল্টো দিকে যাই
3. ফলে loss দ্রুত কমে

4. Gradient Descent এর সাথে সংযোগ (The Bridge)

আগের অংশে আমরা দেখেছি:

Gradient vector (∇L) সবসময় সেই দিকে নির্দেশ করে যেখানে loss সবচেয়ে দ্রুত বাড়ে।

অর্থাৎ:

Gradient = direction of maximum increase

কিন্তু Machine Learning এ আমাদের লক্ষ্য হলো:

Loss কমানো (minimize করা)

তাই যদি gradient দেখায় কোথায় loss বাড়ছে, তাহলে আমাদের যেতে হবে তার উল্টো দিকে।

এই উল্টো দিককে বলা হয়:

$-\nabla L$

এটাকে বলে Negative Gradient।

পাহাড়ের উদাহরণ

ভাবো তুমি পাহাড়ের ঢালে দাঁড়িয়ে আছো।

Gradient বলছে:

কোন দিকে গেলে সবচেয়ে দ্রুত উপরে উঠবে।

কিন্তু আমরা চাই:

নিচে নামতে (minimum error)

তাই আমরা হাঁটবো:

Gradient এর বিপরীত দিকে।

Gradient Descent Update Rule

Gradient Descent এ weights update করার মূল formula হলো:

$w_new = w_old - \eta \nabla L(w_old)$

এখানে তিনটি গুরুত্বপূর্ণ অংশ আছে।

---

1. w_old

এটি হলো:

পুরনো weight

যা বর্তমানে model ব্যবহার করছে।

---

2. ∇L(w_old)

এটি হলো:

Gradient

অর্থাৎ Loss function এর slope।

এটি বলে:

- কোন দিকে গেলে loss বাড়বে
- কত দ্রুত বাড়বে

---

3. η (Eta)

এটাকে বলে:

Learning Rate

এটি একটি ছোট সংখ্যা।

উদাহরণ:

0.1
0.01
0.001

এটি ঠিক করে:

এক ধাপে কতটা move করবো।

Learning Rate এর গুরুত্ব

যদি **learning rate** খুব বড় হয়

সমস্যা:

- model minimum কে overshoot করে ফেলবে
- training unstable হবে

যদি **learning rate** খুব ছোট হয়

সমস্যা:

- training খুব ধীরে হবে
- convergence সময় বেশি লাগবে

Gradient Descent এর Step-by-Step Process

Machine learning training সাধারণত এভাবে হয়:

Step 1
Model prediction করে।

Step 2
Loss calculate করা হয়।

$L(w)$

Step 3
Loss এর gradient বের করা হয়।

$\nabla L(w)$

Step 4
Weights update করা হয়।

$w = w - \eta \nabla L$

Step 5
আবার repeat করা হয়।

এই প্রক্রিয়াকে বলা হয়:

Optimization

কেন এটাকে **Gradient Descent** বলা হয়

Gradient → slope নির্দেশ করে
Descent → নিচে নামা

অর্থাৎ:

Gradient ব্যবহার করে error surface এর নিচে নামা।

Machine Learning এ বাস্তব ব্যবহার

Gradient descent ব্যবহার হয়:

- Linear Regression
- Logistic Regression
- Neural Networks
- Deep Learning models

বিশেষ করে Neural Network training এর মূল ভিত্তি হলো:

- Gradient
- Backpropagation
- Gradient Descent